

Machine Learning

Dr. Irfan Yousuf

Department of Computer Science (New Campus)

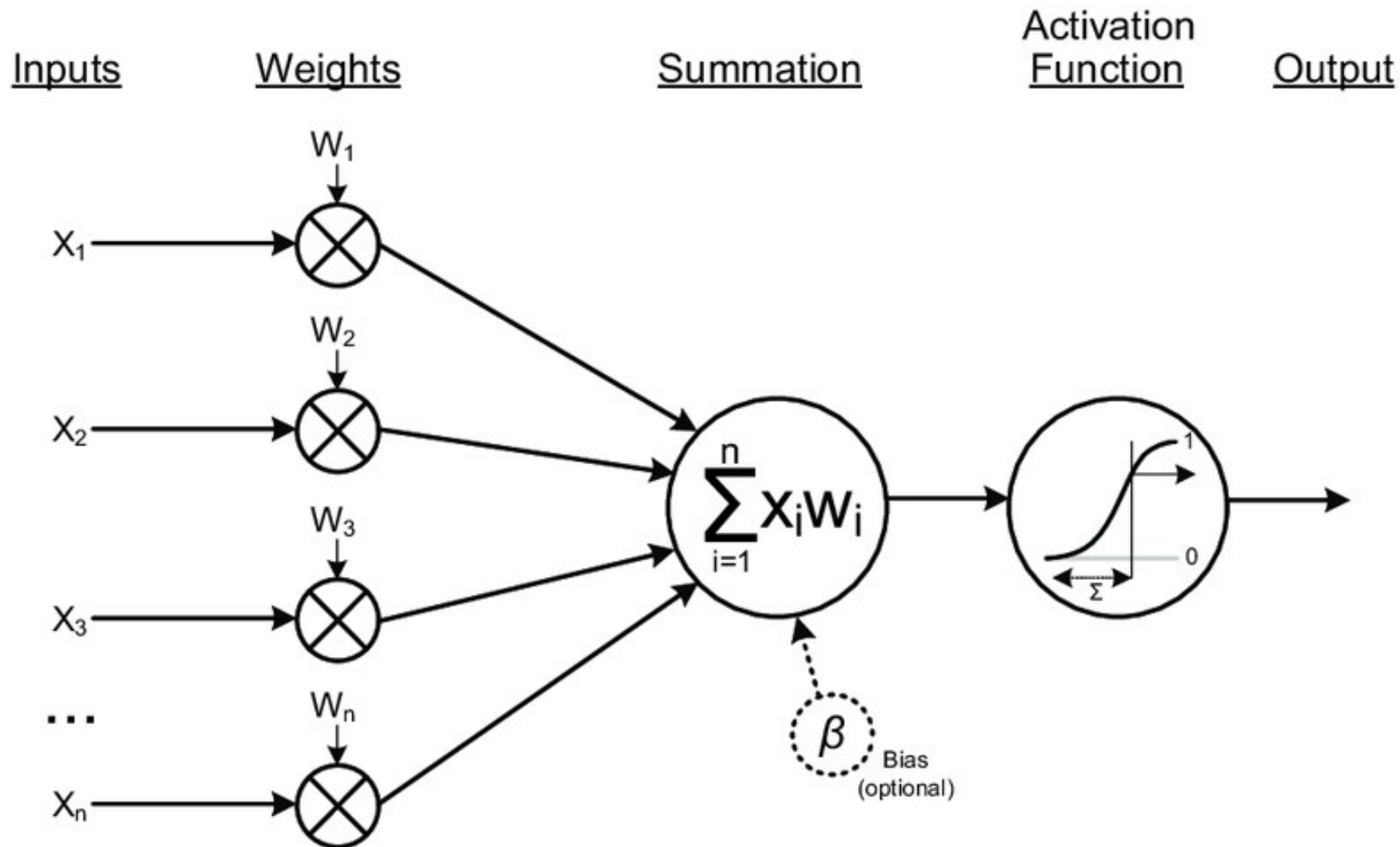
UET, Lahore

(Week 7; October 27 – 31, 2025)

Outline

- Activation Functions
- Feedforward Neural Networks
- Backpropagation

Activation Function in Neural Networks



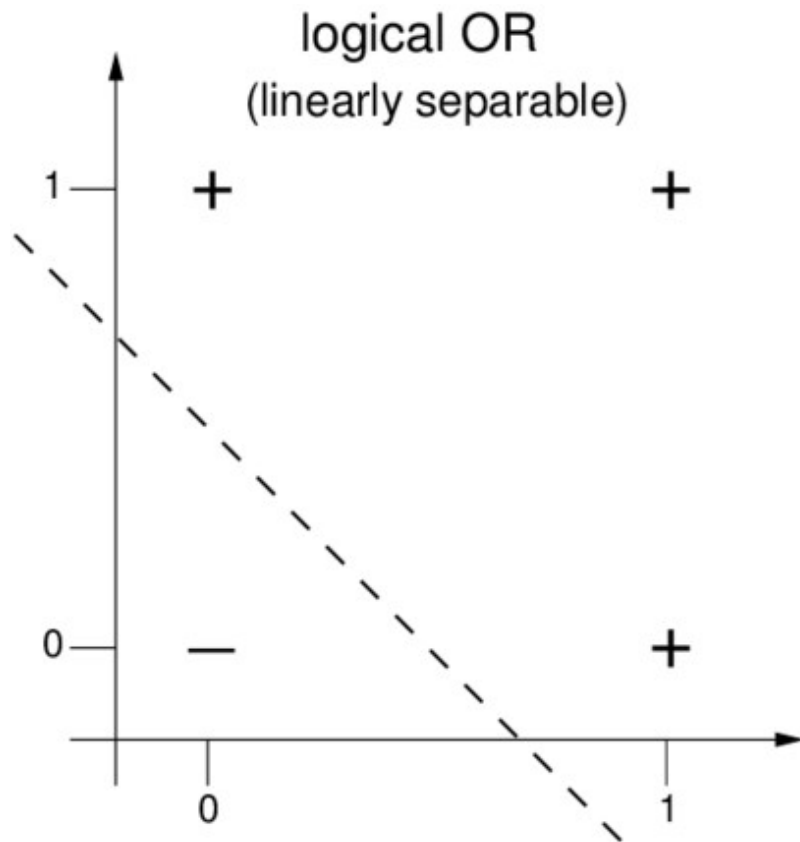
Activation Function in Neural Networks

- An activation function in a neural network defines how the **weighted sum of the input is transformed into an output** from a node or nodes in a layer of the network.
- Activation function decides, **whether a neuron should be activated or not** by calculating weighted sum and further adding bias with it.
- The purpose of the activation function is to **introduce non-linearity** into the output of a neuron.

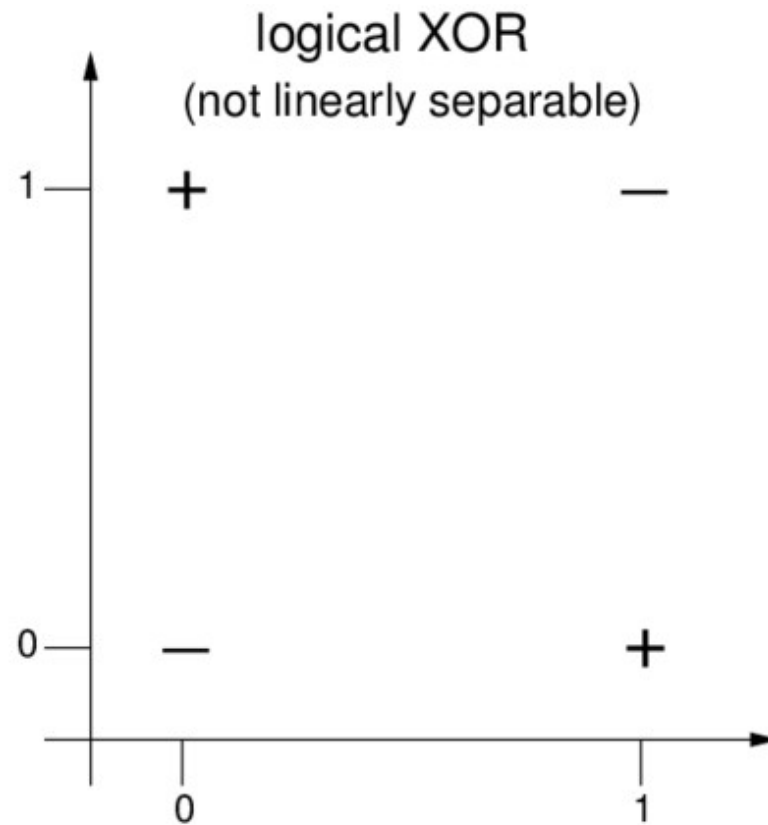
Why do Neural Networks Need an Activation Function?

- The purpose of an activation function is to add **non-linearity** to the neural network.
- Let's suppose we have a neural network working without the activation functions.
- In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases.
- It's because it doesn't matter how many hidden layers we attach in the neural network; all layers will behave in the same way because the composition of two linear functions is a linear function itself.

Why do Neural Networks Need an Activation Function?



Input ₁	Input ₂	Output
1	1	1
1	0	1
0	1	1
0	0	0



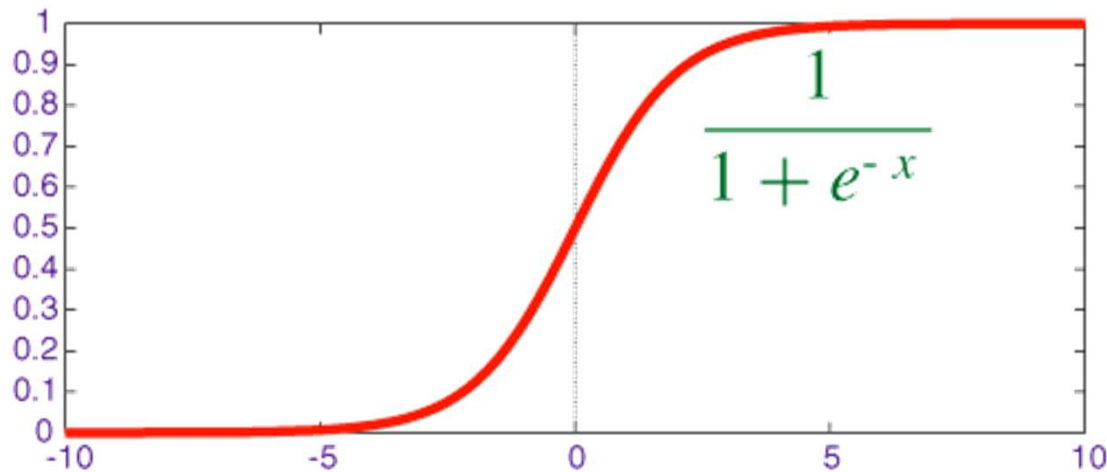
Input ₁	Input ₂	Output
1	1	0
1	0	1
0	1	1
0	0	0

Activation Functions in Neural Networks

- Logistic (Sigmoid)
- Hyperbolic Tangent
- Unit Step
- Sign (Signum)
- Linear
- Rectifier (ReLU) Rectifier Linear Unit
- Softmax

Logistic (Sigmoid)

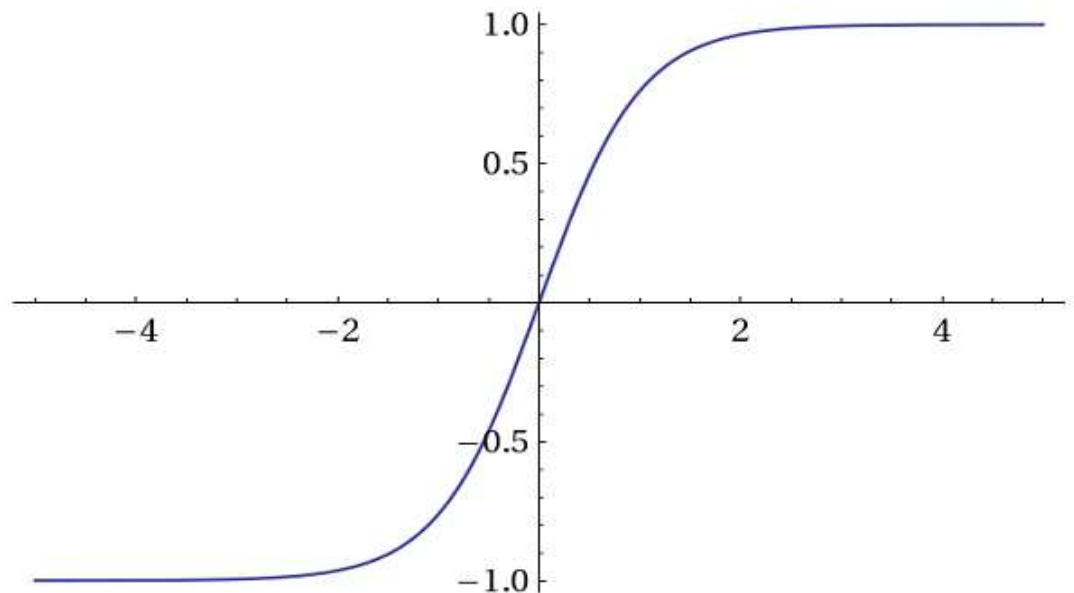
- Sigmoid is an activation function which is in S shape curve. It takes real value as an input and gives the output which is in between 0 and 1.
- It is **non-linear in nature**; it is continuously differentiable and has fixed output range of values.



Hyperbolic Tangent

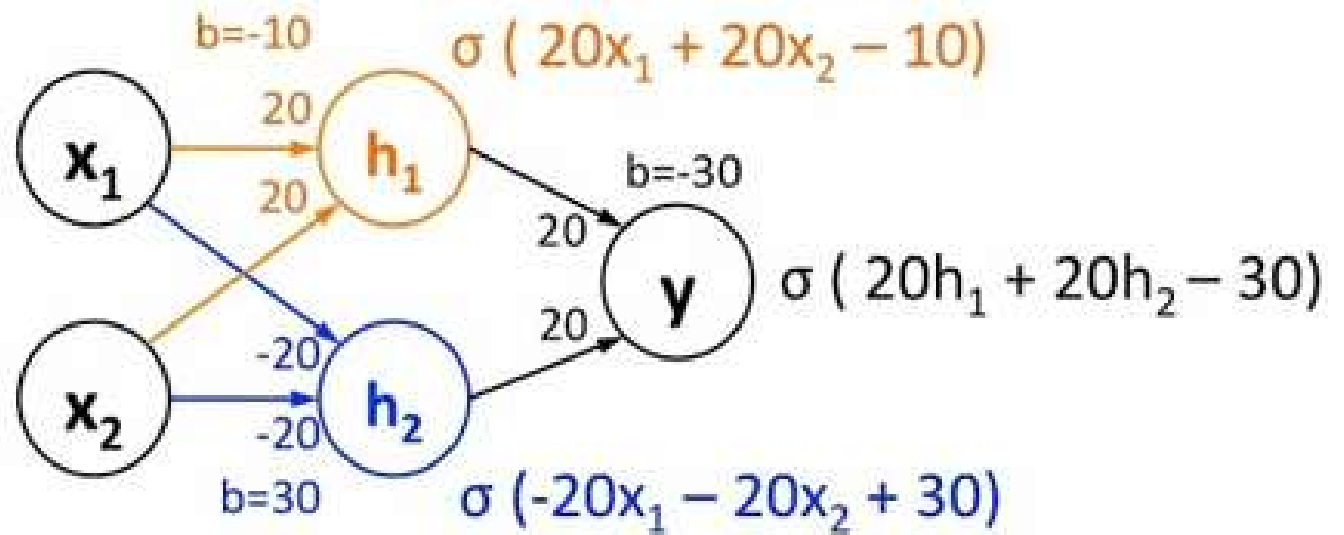
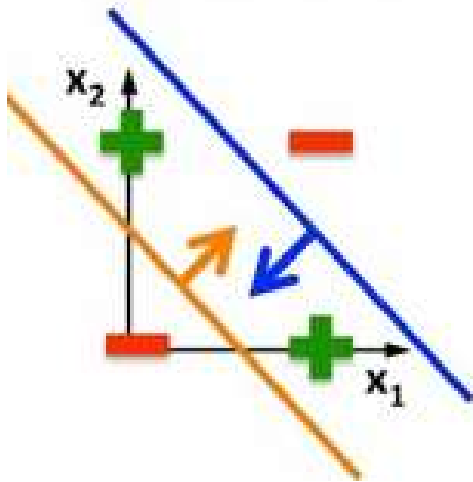
- The hyperbolic tangent activation function is also referred to simply as the Tanh (also “tanh” and “TanH”) function.
- It is very similar to the sigmoid activation function and even has the same S-shape.
- The function takes any real value as input and outputs values in the range -1 to 1.

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



XOR Problem

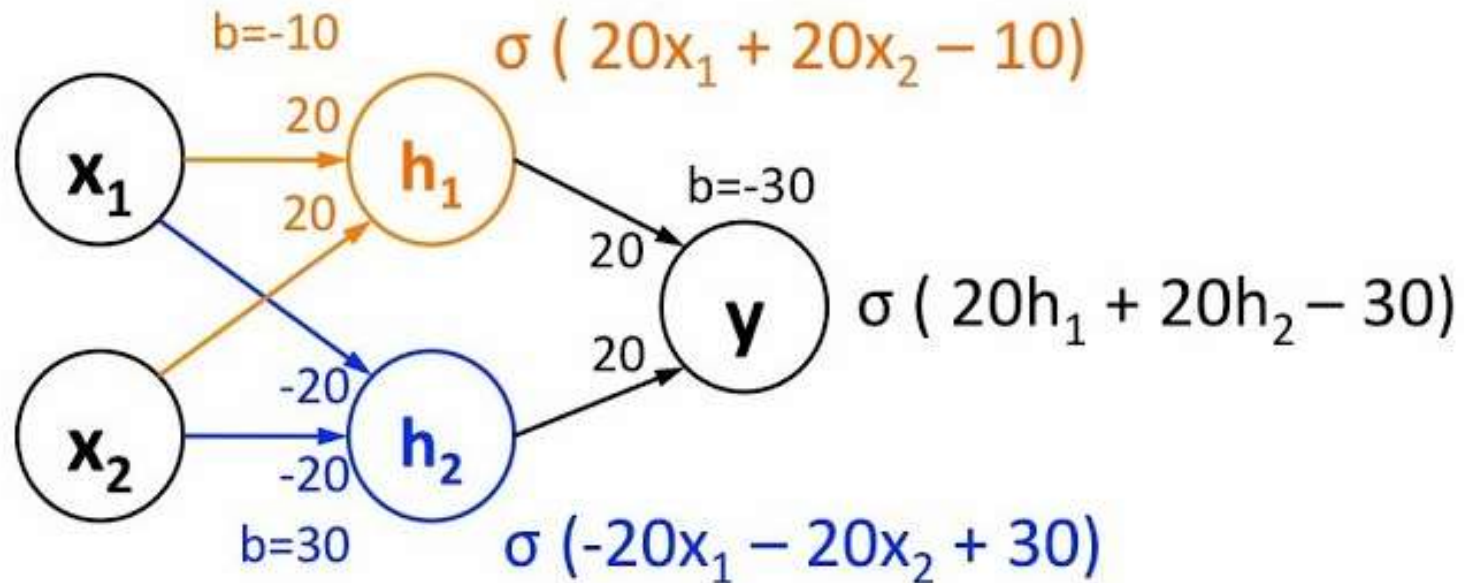
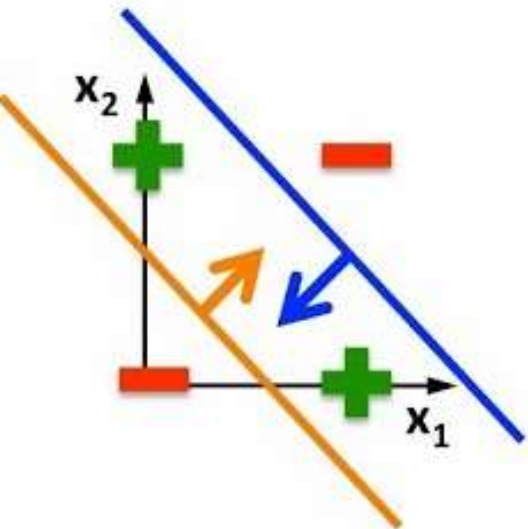
Linear classifiers
cannot solve this



Activation Functions at hidden and output layers are Sigmoid and Hyperbolic Tangent respectively.

XOR Problem

Linear classifiers
cannot solve this



$$\sigma(20*0 + 20*0 - 10) \approx 0$$

$$\sigma(20*1 + 20*1 - 10) \approx 1$$

$$\sigma(20*0 + 20*1 - 10) \approx 1$$

$$\sigma(20*1 + 20*0 - 10) \approx 1$$

$$\sigma(-20*0 - 20*0 + 30) \approx 1$$

$$\sigma(-20*1 - 20*1 + 30) \approx 0$$

$$\sigma(-20*0 - 20*1 + 30) \approx 1$$

$$\sigma(-20*1 - 20*0 + 30) \approx 1$$

$$\sigma(20*0 + 20*1 - 30) \approx 0$$

$$\sigma(20*1 + 20*0 - 30) \approx 0$$

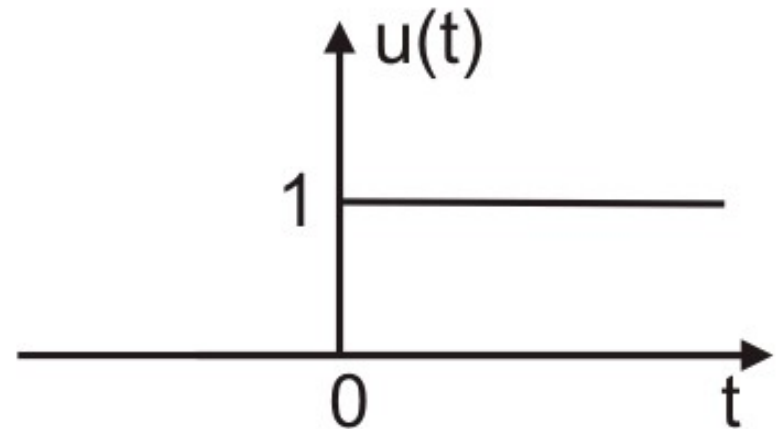
$$\sigma(20*1 + 20*1 - 30) \approx 1$$

$$\sigma(20*1 + 20*1 - 30) \approx 1$$

Unit Step

- A unit step activation function is a much-used function in neural networks.
- The output assumes **value 0 for negative argument and 1 for positive argument.**
- Also called Heaviside Function

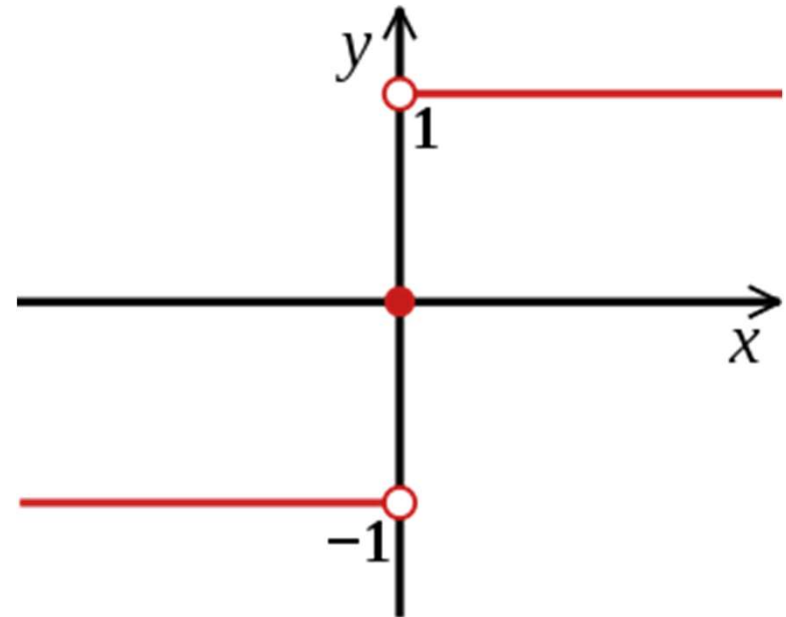
$$u(t) = \begin{cases} 0 & \text{for } t < 0 \\ 1 & \text{for } t \geq 0 \end{cases}$$



Sign (Signum)

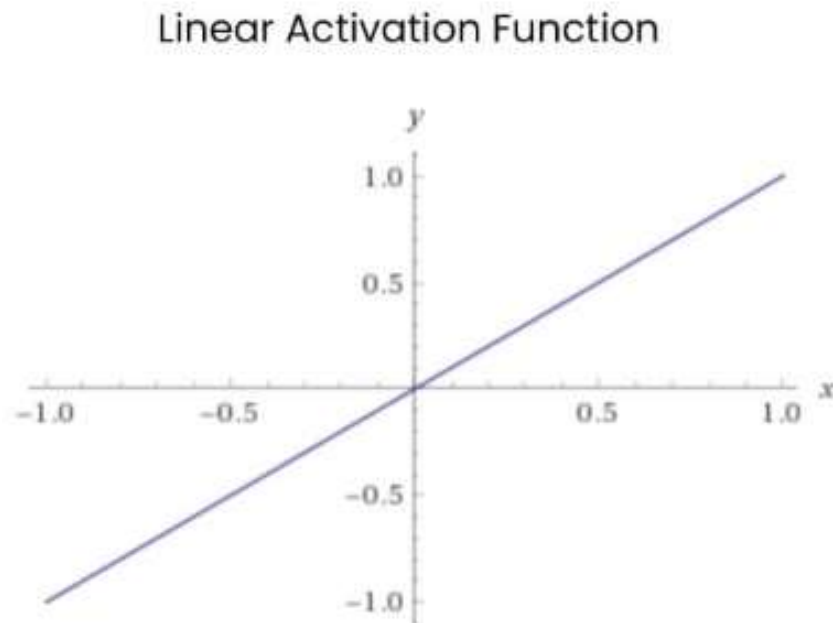
- In mathematics, the sign function or signum function is a mathematical function that extracts the sign of a real number.

$$\operatorname{sgn} x := \begin{cases} -1 & \text{if } x < 0, \\ 0 & \text{if } x = 0, \\ 1 & \text{if } x > 0. \end{cases}$$



Linear

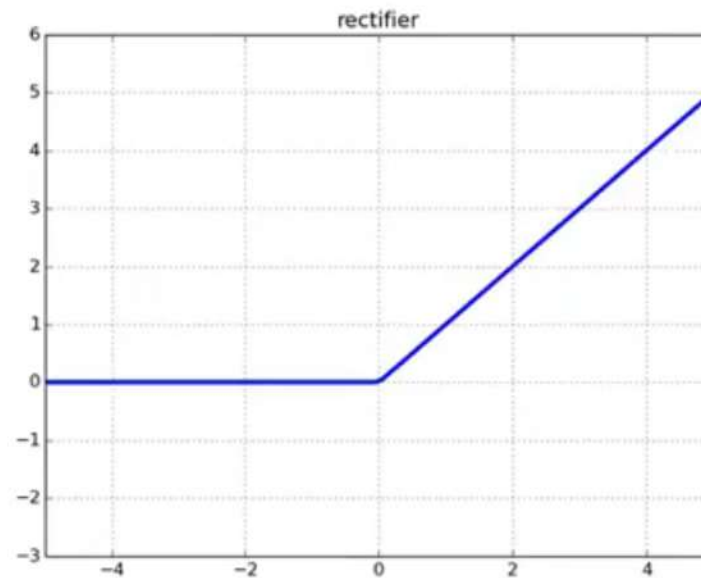
- The linear activation function simply returns **the neuron input value as its output value**, without manipulating it at all.
- This type of activation function is often used where regression is to be applied, and the network is to learn to output numeric values.



Rectifier Linear Unit (ReLU)

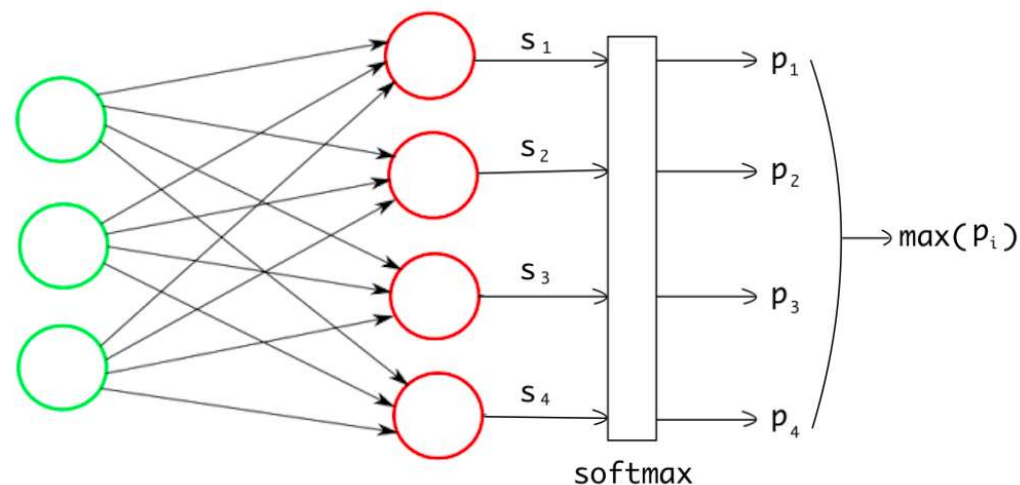
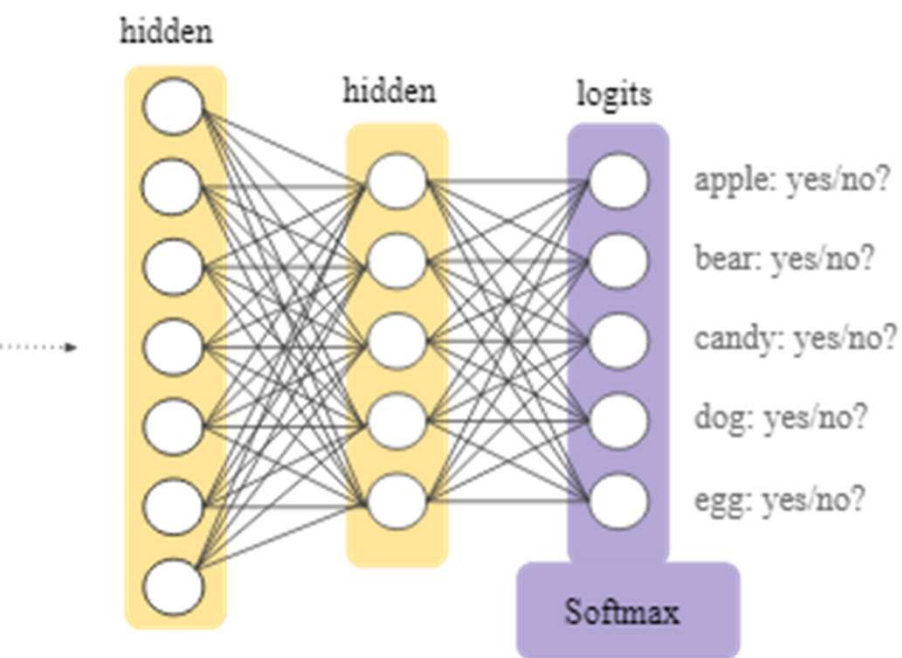
- ReLU stands for Rectified Linear Unit.
- Mathematically, it is defined as $y = \max(0, x)$.

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$



Softmax

- It is used in **multinomial logistic regression** and is often used as the last activation function of a neural network to normalize the output of a network to a **probability distribution** over predicted output classes.



Softmax

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax

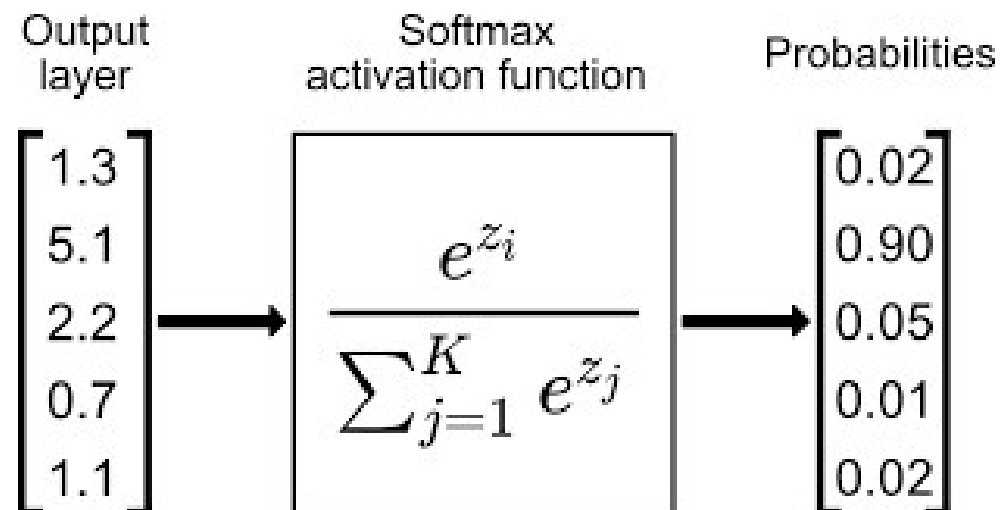
$\vec{z}$ = input vector

e^{z_i} = standard exponential function for input vector

K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

e^{z_j} = standard exponential function for output vector

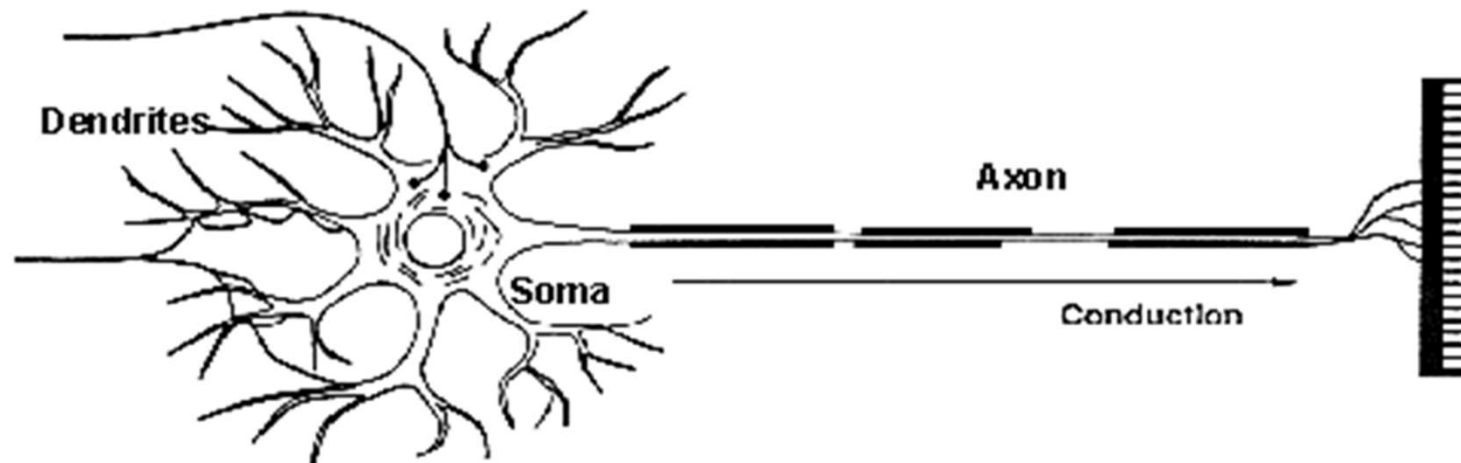


How to choose Activation Function

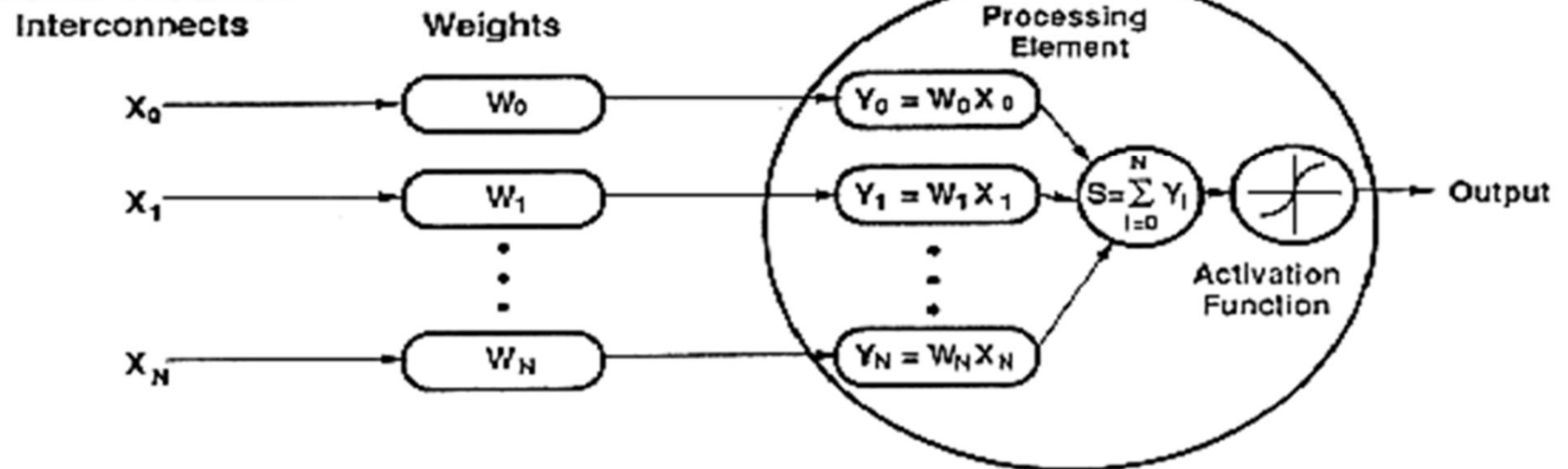
- **Unit Step** is useful when the input pattern can only belong to **one or two groups**, that is, binary classification.
- **Sigmoid** functions and their combinations generally work better in the case of **classifiers**.
- **Tanh** is usually used in hidden layers of a neural network as it's values lies between -1 to 1 hence the mean for the hidden layer comes out be 0 or very close to it, hence helps in **centering the data** by bringing mean close to 0.
- **Softmax** is used for multi-class classification.

Neural Networks

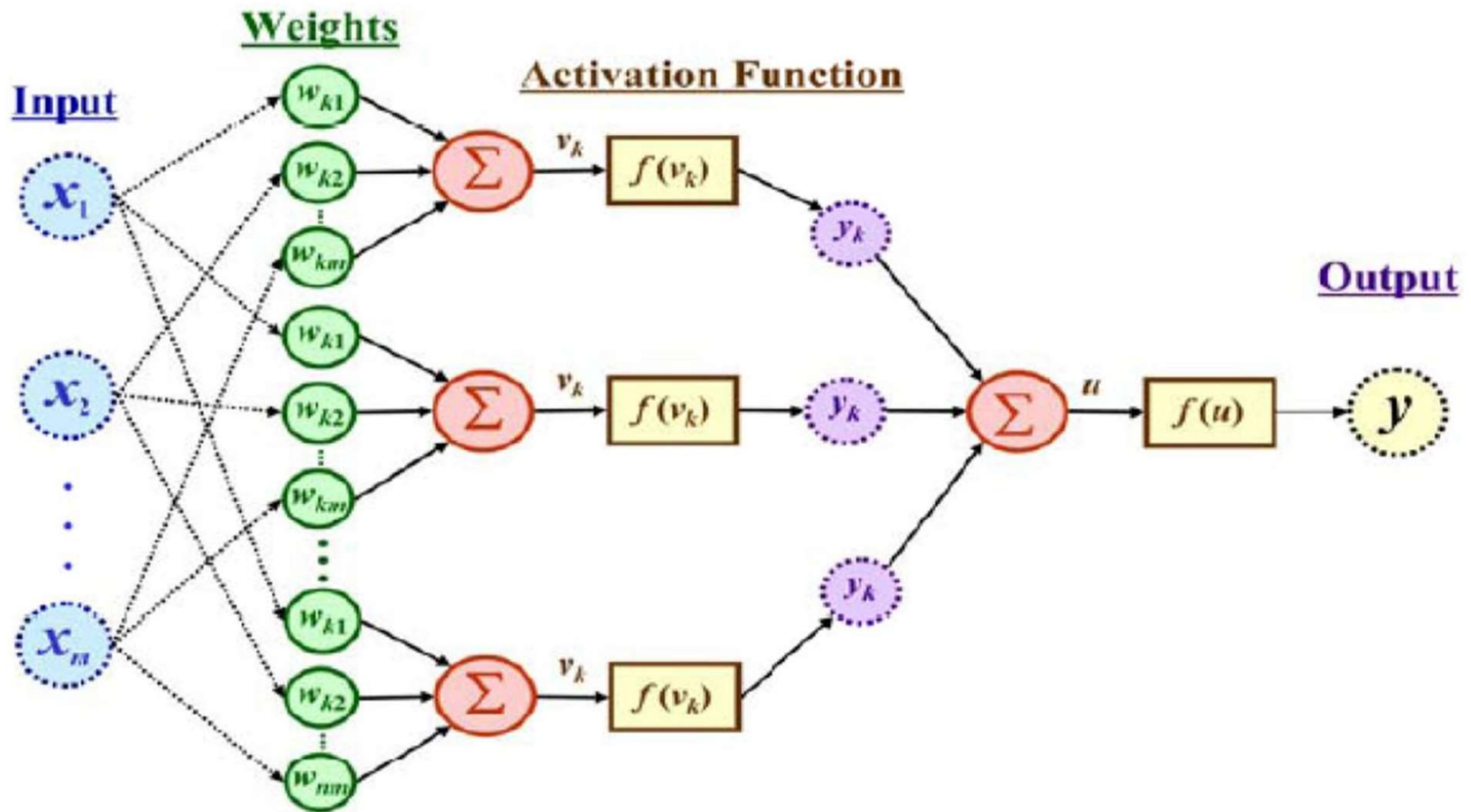
Biological Neuron



Artificial Neuron



Neural Networks



Artificial Neural Networks

- An ANN can:
 - compute any computable function, by the appropriate selection of the network topology and weights values.
 - learn from experience!
 - Specifically, by trial-and-error

Learning by trial-and-error

- Continuous process of:
- **Trial:**
 - Processing an input to produce an output (In terms of ANN: Compute the output function of a given input)
- **Evaluate:**
 - Evaluating this output by comparing the actual output with the expected output.
- **Adjust:**
 - Adjust the weights

Artificial Neural Networks

- A neural network is characterized by
 - Its pattern of connections between the neurons (called its architecture).
 - Its method of determining the weights on the connections (called its training, or learning, algorithm).
 - Its activation function.

Backpropagation in NN

- Iteratively process a set of training tuples & compare the network's prediction with the actual known target value
- For each training tuple, the weights are modified to minimize the mean squared error between the network's prediction and the actual target value
- Modifications are made in the “backwards” direction: from the output layer, through each hidden layer down to the first hidden layer, hence “backpropagation”.

Backpropagation in NN

- Initialize weights (to small random #s) and biases in the network
- Propagate the inputs forward (by applying activation function)
- Backpropagate the error (by updating weights and biases)
- Terminating condition (when error is very small, etc.)

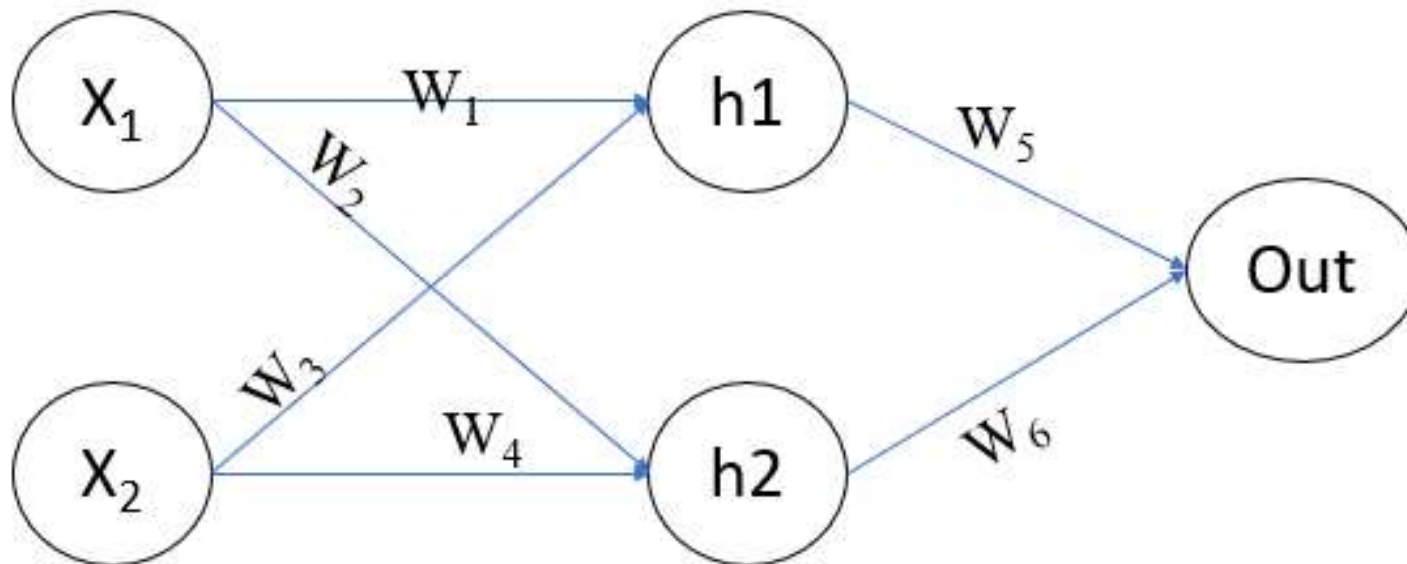
Backpropagation in NN

- The backpropagation algorithm is the method for fine-tuning the weights of a neural network with respect to the error rate obtained in the previous iteration or epoch, and this is a standard method of training artificial neural networks.
- We can think of it as a feedback system where, after each round of training or 'epoch,' the network reviews its performance on tasks.
- It calculates the difference between its output and the correct answer, known as the error. Then, it adjusts its internal parameters, or 'weights,' to reduce this error next time

How Does Backpropagation Work?

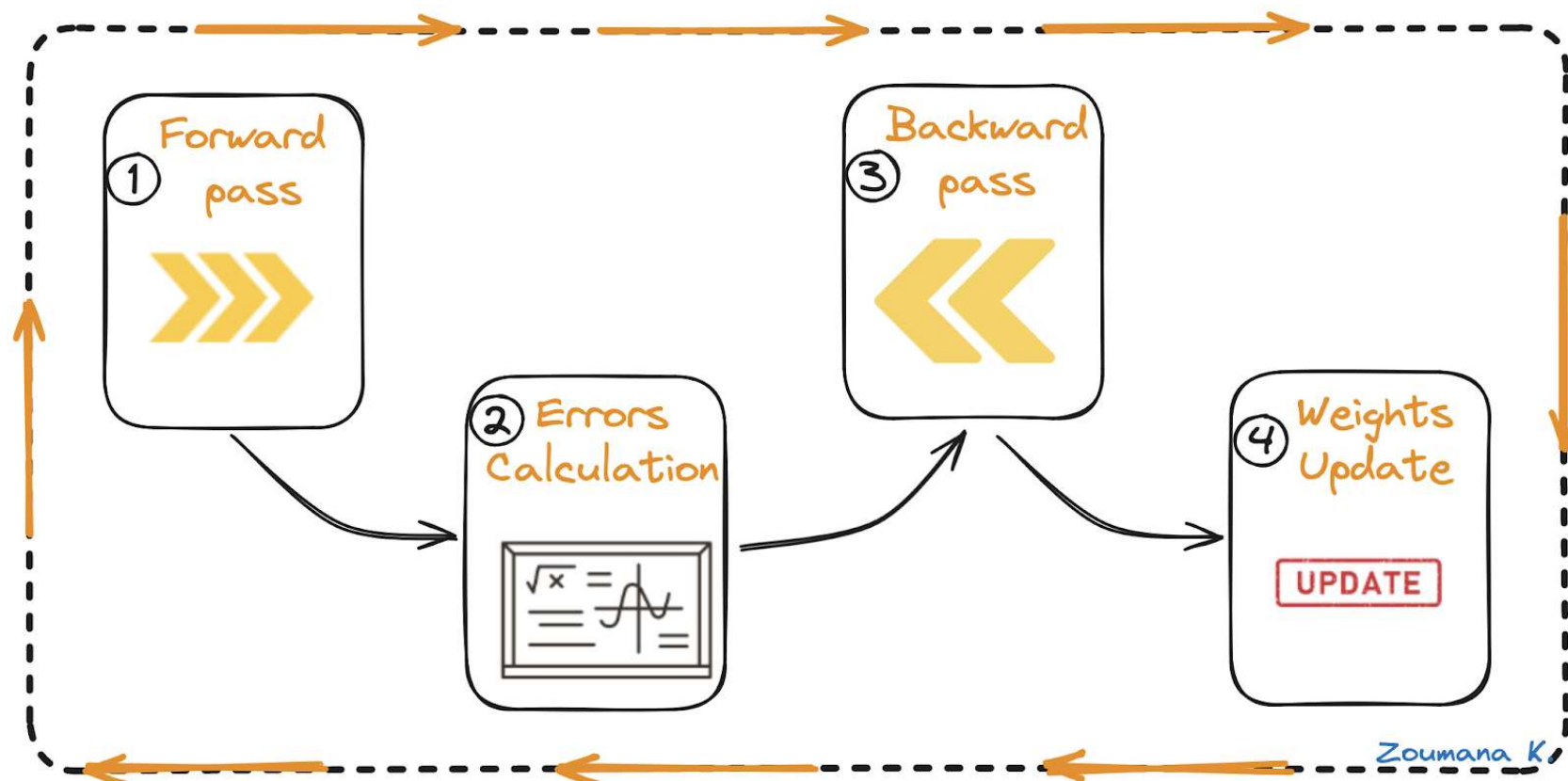
Consider a NN with

- Two inputs at Input Layer
- One hidden layer with two Neurons
- One output layer with 1 Neuron



How Does Backpropagation Work?

- Forward pass
- Errors calculation
- Backward pass
- Weights update



How Does Backpropagation Work?

- **Forward pass**
- The data (inputs X_1 and X_2) is fed to the input layer
- Then, each input is multiplied by its corresponding weight, and the results are passed to the neurons of the hidden layers.
- Those neurons apply an activation function to the weighted inputs they receive, and the result passes to the next layer.

How Does Backpropagation Work?

- **Error calculation**
- The process continues until the output layer generates the final output (o/p).
- The output of the network is then compared to the ground truth (desired output), and the difference is calculated, resulting in an error value.

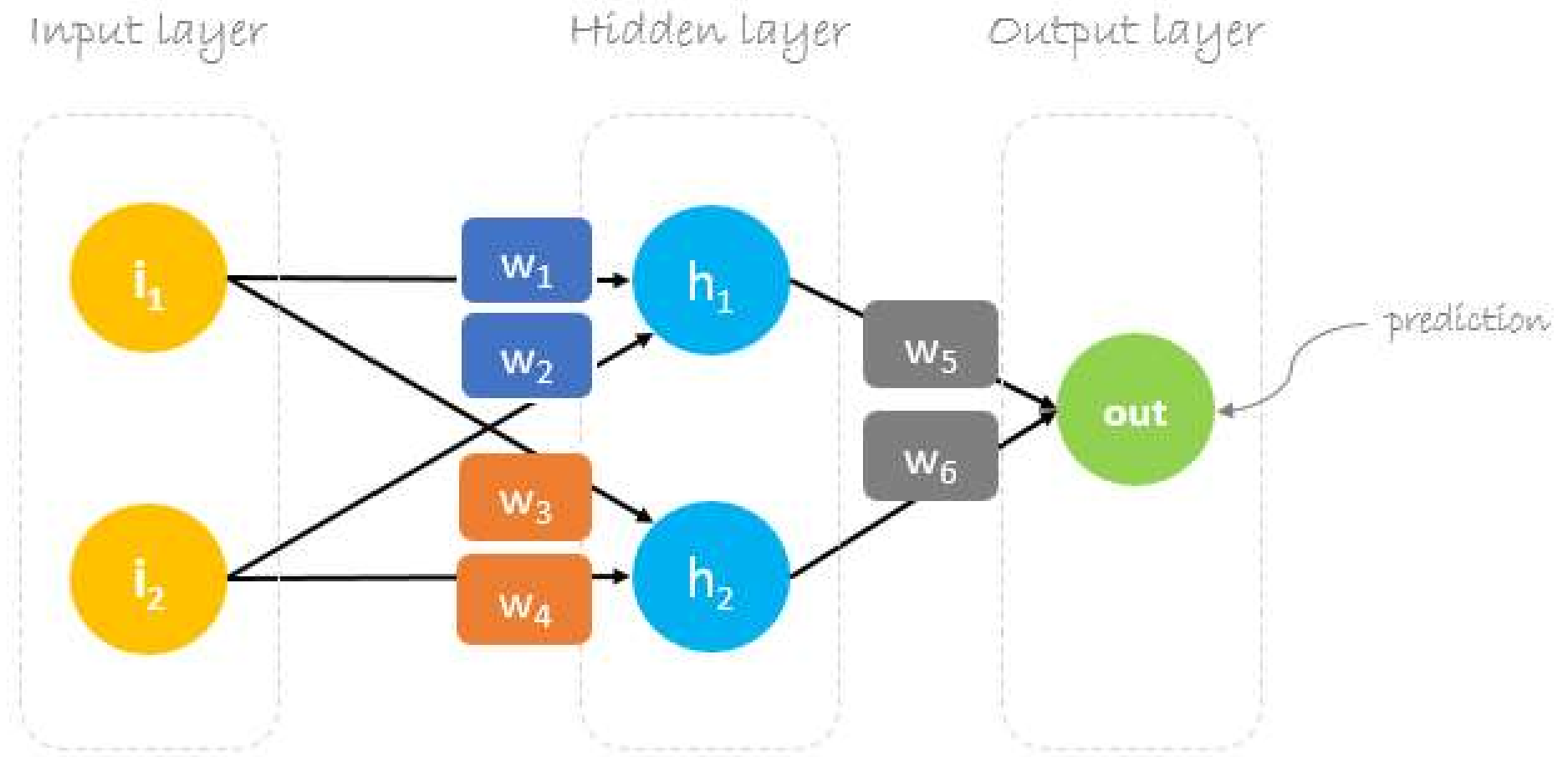
How Does Backpropagation Work?

- **Backward Pass**
- The error value obtained previously is used to calculate the **gradient of the loss function**.
- The gradient of the error is propagated back through the network, starting from the output layer to the hidden layers.
- As the error gradient propagates back, the weights are updated according to their contribution to the error. This involves taking the **derivative of the error with respect to each weight**, which indicates how much a change in the weight would change the error.
- The **learning rate determines the size of the weight updates**. A smaller learning rate means that the weights are updated by a smaller amount, and vice-versa.

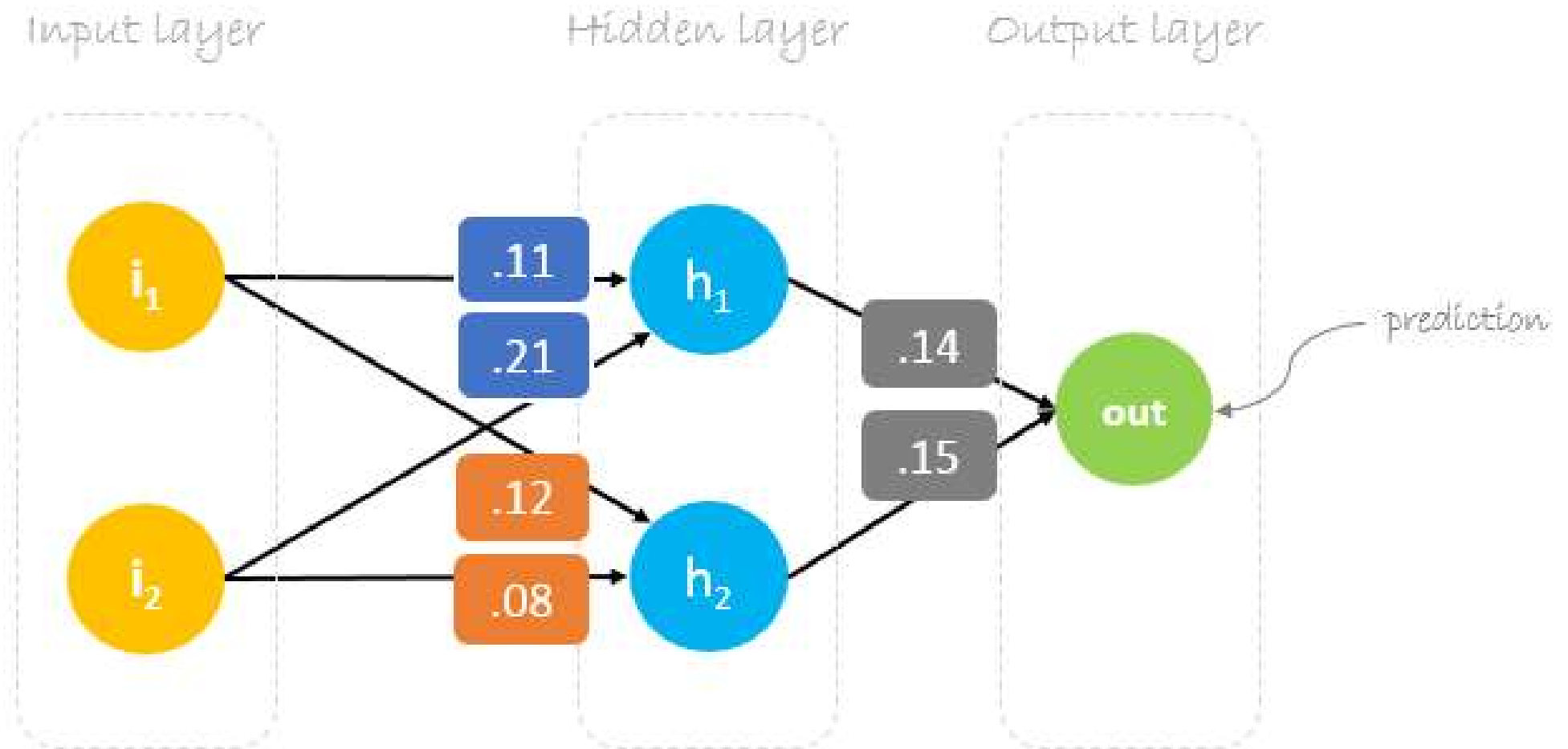
How Does Backpropagation Work?

- **Weight Updates**
- The weights are updated in the opposite direction of the gradient, leading to the name “gradient descent.” It aims to reduce the error in the next forward pass.
- This process of forward pass, error calculation, backward pass, and weights update continues for multiple epochs until the network performance reaches a satisfactory level or stops improving significantly.

Backpropagation: Example

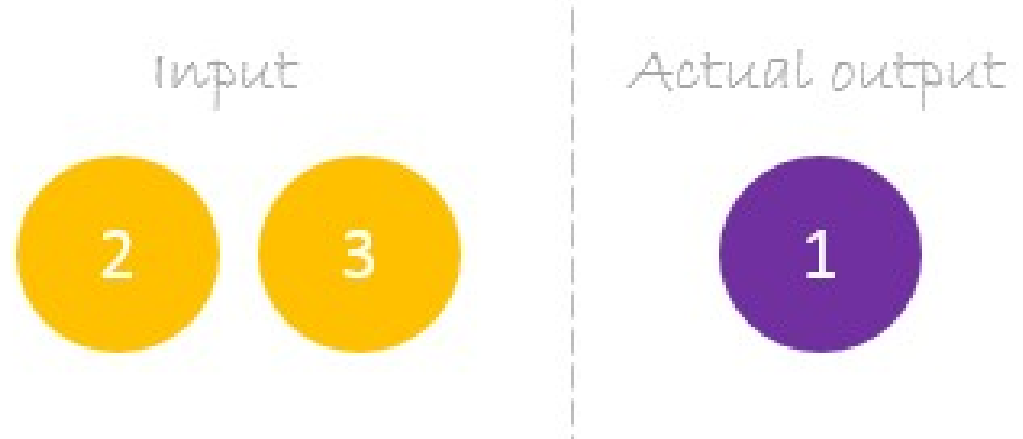


Backpropagation: Example

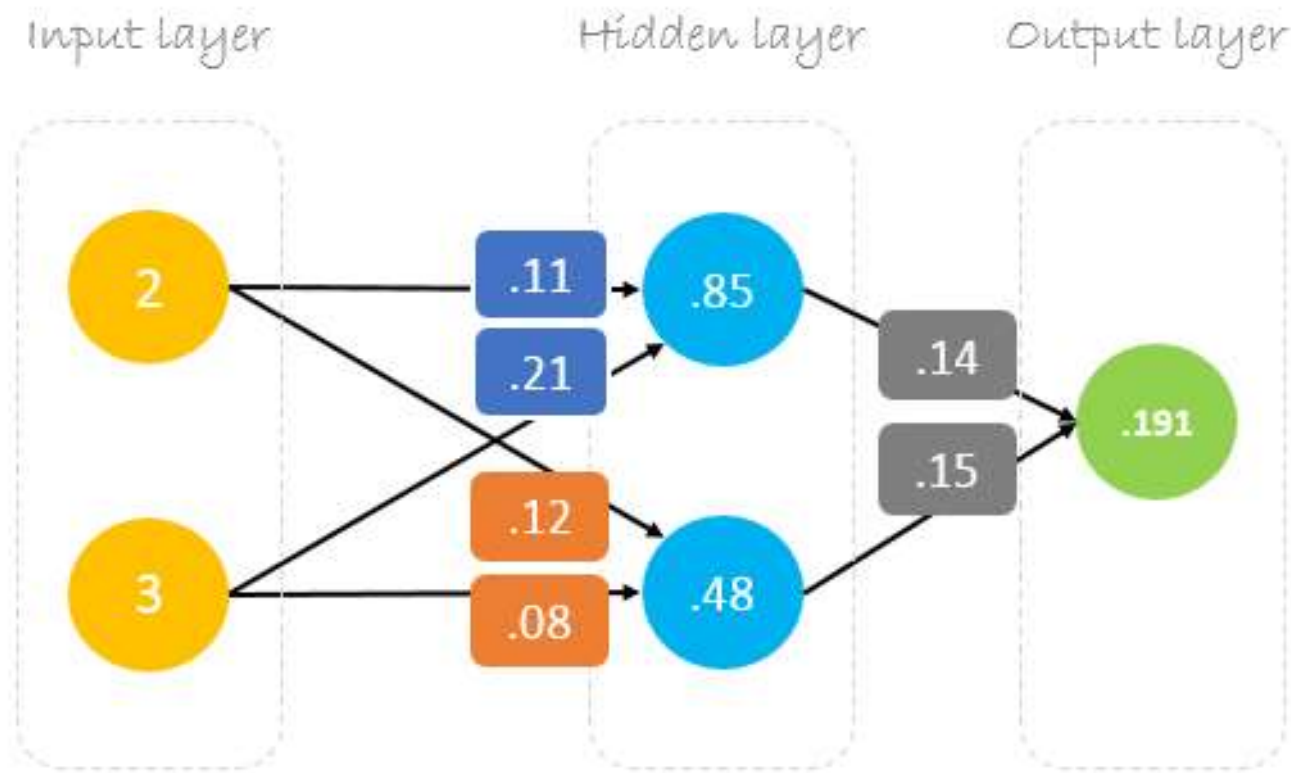


Backpropagation: Example

Our single sample is as following `inputs=[2, 3]` and `output=[1]` .



Backpropagation: Example



Forward Pass

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = \begin{bmatrix} 0.85 & 0.48 \end{bmatrix} \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.191 \end{bmatrix}$$

Matrix multiplication

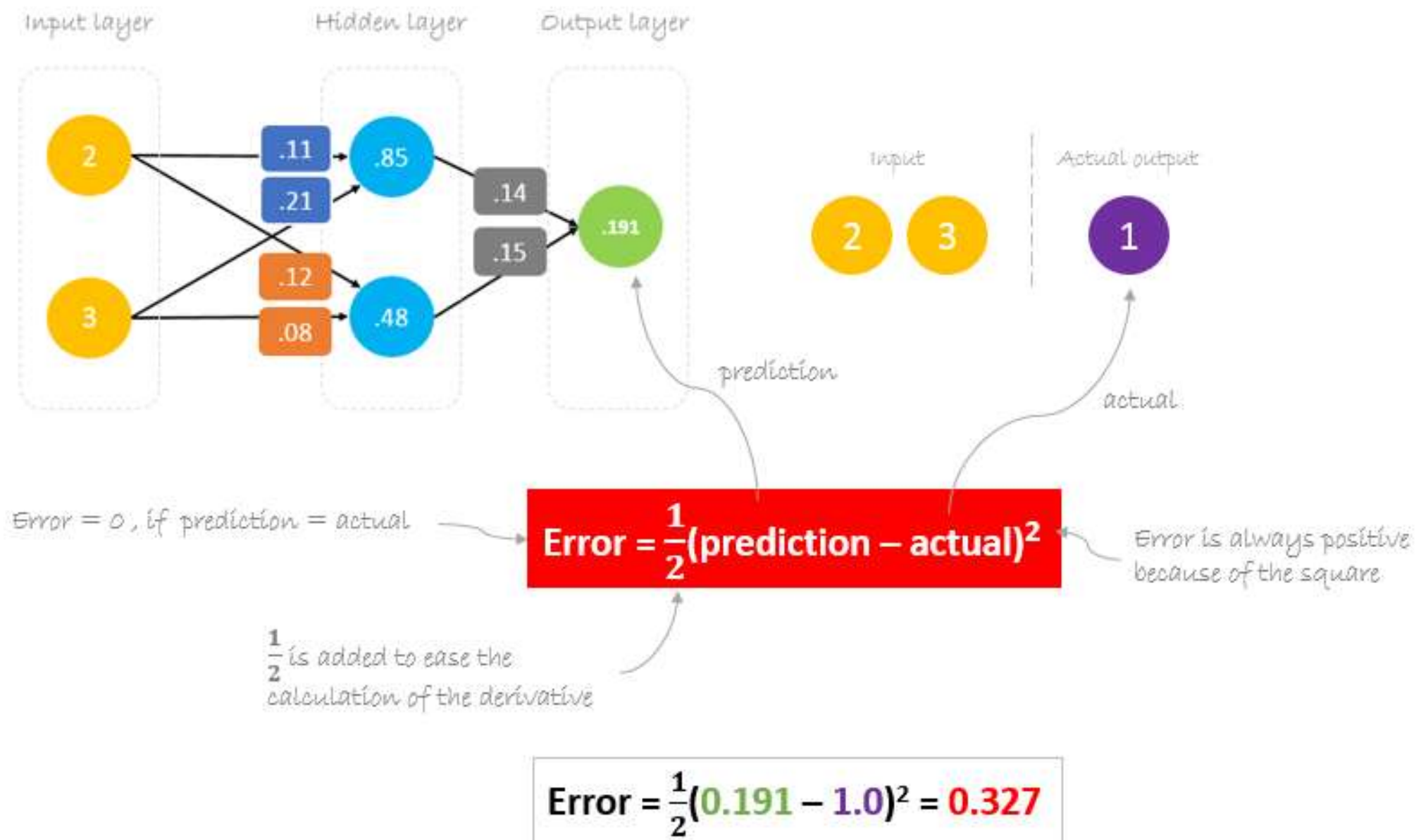
Details

$$2 \times .11 + 3 \times .21 = .85$$

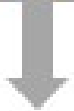
$$.85 \times .14 + .48 \times .15 = .191$$

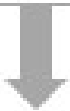
$$2 \times .12 + 3 \times .08 = .48$$

Backpropagation: Example



Backpropagation: Example

$$\text{prediction} = \text{out}$$


$$\text{prediction} = \frac{(h_1) w_5 + (h_2) w_6}{}$$


$$\begin{aligned} h_1 &= i_1 w_1 + i_2 w_2 \\ h_2 &= i_1 w_3 + i_2 w_4 \end{aligned}$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

to change *prediction* value,
we need to change *weights*

Backpropagation: Example

- Backpropagation, short for “backward propagation of errors”, is a mechanism used to update the weights using **gradient descent**.
- It calculates the gradient of the error function with respect to the neural network’s weights. The calculation proceeds backwards through the network.

Backpropagation: Example

- **Gradient descent is an iterative optimization algorithm** for finding the minimum of a function; in our case we want to minimize the error function.
- To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.

Backpropagation: Example

$$X = X - lr * \frac{d}{dX} f(X)$$

Where,

X = *input*

$F(X)$ = *output based on X*

lr = *learning rate*

Backpropagation: Example

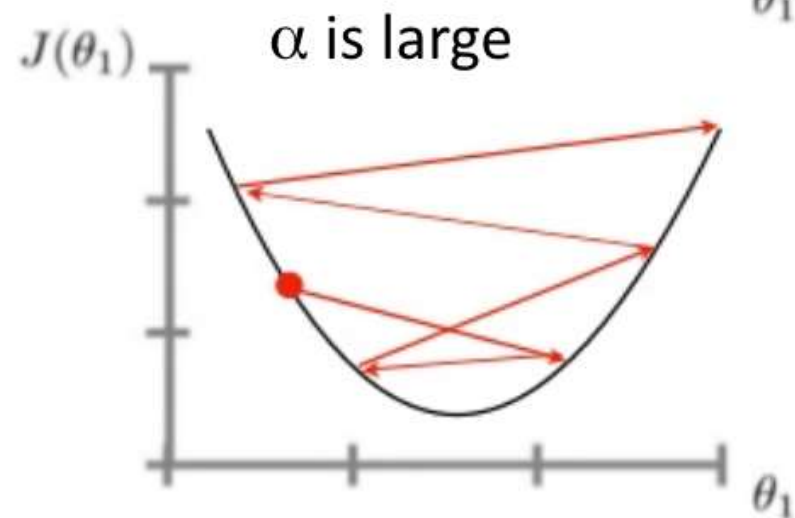
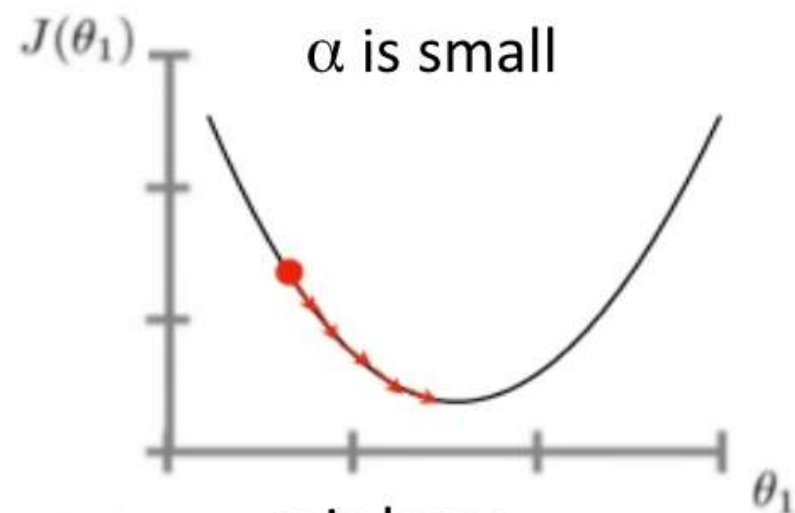
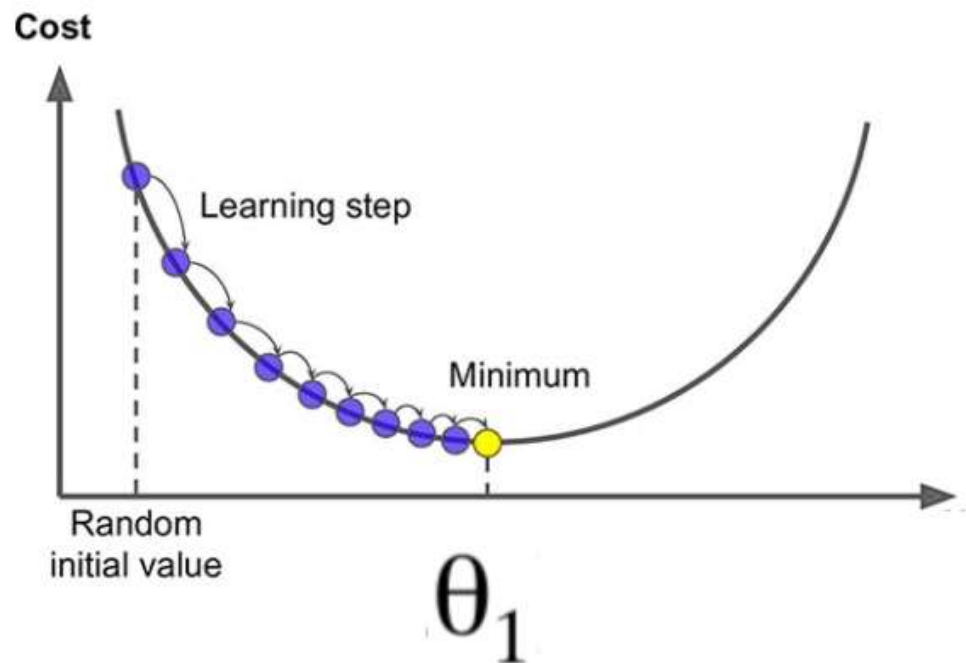
The diagram illustrates the weight update equation for gradient descent. It features the equation $w := w - \eta \nabla Q_i(w)$ with color-coded components and labels. The new weight w is in a green box, labeled 'new weight'. The previous weight w is in a green box, labeled 'previous weight'. The learning rate η is in a yellow box, labeled 'learning rate'. The gradient of the loss function $\nabla Q_i(w)$ is in a blue box, labeled 'gradient of the loss function'. A bracket under the $\eta \nabla Q_i(w)$ term is labeled 'gradient descent'.

$$\boxed{w} := \boxed{w} - \boxed{\eta} \boxed{\nabla Q_i(w)}$$

gradient descent

Backpropagation: Example

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}



Backpropagation: Example

$$y = f(g(x))$$

$$y' = f'(g(x)) \cdot g'(x)$$

(Ex)

$$y = (4x^2 - 2x)^3$$

"Derivative of the outside
times
the derivative of the inside"

$$f(g(x)) = (\text{something})^3 \quad g(x) = 4x^2 - 2x$$

$$f'(g(x)) = 3(4x^2 - 2x)^2 \quad g'(x) = 8x - 2$$

$\Downarrow$

$$y' = 3(4x^2 - 2x)^2 \cdot (8x - 2)$$

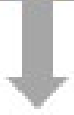
At

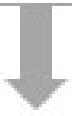
Backpropagation: Example

If we have $y = f(u)$ and $u = g(x)$ then we can write the derivative of y as:

$$\frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dx}$$

Backpropagation: Example

$$\text{prediction} = \text{out}$$


$$\text{prediction} = \frac{(h_1) w_5 + (h_2) w_6}{}$$


$$\begin{aligned} h_1 &= i_1 w_1 + i_2 w_2 \\ h_2 &= i_1 w_3 + i_2 w_4 \end{aligned}$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

to change *prediction* value,
we need to change *weights*

Backpropagation: Example

The diagram illustrates the weight update formula in backpropagation. The equation is $*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$. Annotations include: 'Old weight' pointing to W_x , 'Derivative of Error with respect to weight' pointing to $\left(\frac{\partial \text{Error}}{\partial W_x} \right)$, 'New weight' pointing to $*W_x$, and 'Learning rate' pointing to the yellow coefficient a .

$$*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

Annotations:

- Old weight (points to W_x)
- Derivative of Error with respect to weight (points to $\left(\frac{\partial \text{Error}}{\partial W_x} \right)$)
- New weight (points to $*W_x$)
- Learning rate (points to a)

Backpropagation: Example

$$*W_6 = W_6 - \mathbf{a} \left(\frac{\partial \text{Error}}{\partial W_6} \right)$$

chain rule

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial W_6}$$

Error = $\frac{1}{2}(\text{prediction} - \text{actual})^2$

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{1}{2}(\text{prediction} - \text{actual})^2 * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

prediction = $(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$

$$\frac{\partial \text{Error}}{\partial W_6} = 2 * \frac{1}{2}(\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial \text{Error}}{\partial W_6} = (\text{prediction} - \text{actual}) * (h_2)$$

$\Delta = \text{prediction} - \text{actual}$ *delta*

$$\frac{\partial \text{Error}}{\partial W_6} = \Delta h_2$$

Backpropagation: Example

Similarly, we can derive the update formula for w_5 and any other weights existing between the output and the hidden layer.

$$*W_5 = W_5 - a \Delta h_1$$

Backpropagation: Example

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial h_1} * \frac{\partial h_1}{\partial W_1} \quad \leftarrow \text{chain rule}$$

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \frac{1}{2}(\text{prediction} - \text{actual})^2}{\partial \text{prediction}} * \frac{\partial (h_1) w_5 + (h_2) w_6}{\partial h_1} * \frac{\partial i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial W_1} = 2 * \frac{1}{2} (\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (w_5) * (i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = (\text{prediction} - \text{actual}) * (w_5 i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = \Delta w_5 i_1$$

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$\Delta = \text{prediction} - \text{actual} \quad \leftarrow \text{delta}$$

Backpropagation: Example

updated weights

$$*w_6 = w_6 - a (h_2 \cdot \Delta)$$

$$*w_5 = w_5 - a (h_1 \cdot \Delta)$$

$$*w_4 = w_4 - a (i_2 \cdot \Delta w_6)$$

$$*w_3 = w_3 - a (i_1 \cdot \Delta w_6)$$

$$*w_2 = w_2 - a (i_2 \cdot \Delta w_5)$$

$$*w_1 = w_1 - a (i_1 \cdot \Delta w_5)$$

Backpropagation: Example

$$\begin{aligned}
 *w_6 &= w_6 - a(h_2 \cdot \Delta) \\
 *w_5 &= w_5 - a(h_1 \cdot \Delta) \\
 *w_4 &= w_4 - a(i_2 \cdot \Delta w_6) \\
 *w_3 &= w_3 - a(i_1 \cdot \Delta w_6) \\
 *w_2 &= w_2 - a(i_2 \cdot \Delta w_5) \\
 *w_1 &= w_1 - a(i_1 \cdot \Delta w_5)
 \end{aligned}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} ah_1 \Delta \\ ah_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} ai_1 \Delta w_5 & ai_1 \Delta w_6 \\ ai_2 \Delta w_5 & ai_2 \Delta w_6 \end{bmatrix}$$

Backpropagation: Example

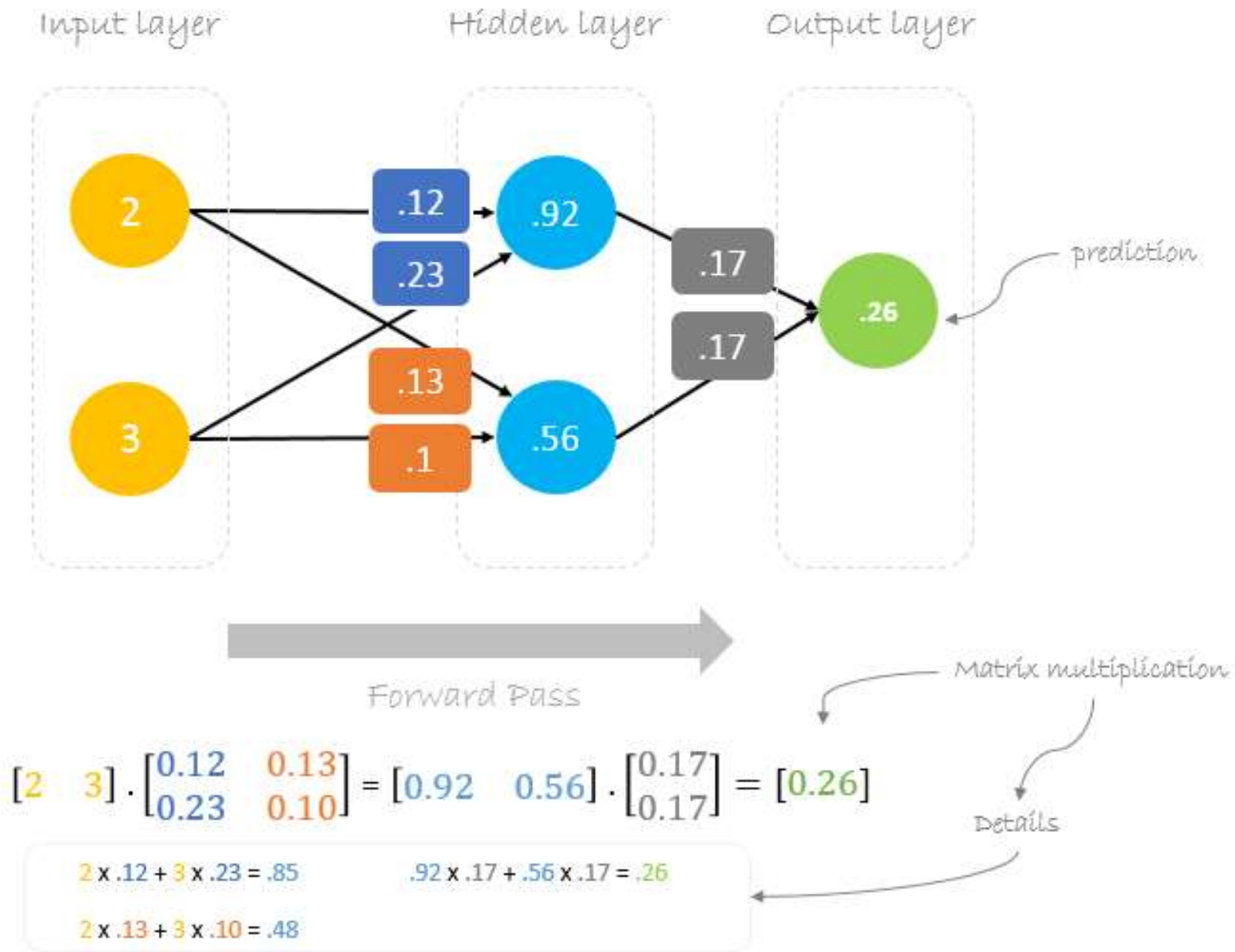
$$\Delta = 0.191 - 1 = -0.809 \quad \leftarrow \text{Delta} = \text{prediction} - \text{actual}$$

$$a = 0.05 \quad \leftarrow \text{Learning rate, we smartly guess this number}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

Backpropagation: Example



Backpropagation: Example

<https://hmkcode.com/ai/backpropagation-step-by-step/>

<https://theneuralblog.com/forward-pass-backpropagation-example/>

Advantages of Backpropagation

- **Ease of implementation:** accessible through multiple deep learning libraries such as Pytorch and Keras, facilitating its use in diverse applications.
- **Programming simplicity:** simplified coding with framework abstraction, reducing the need for complex mathematics.
- **Flexibility:** adaptable to diverse architectures, suitable for a broad spectrum of AI challenges.

Limitations of Backpropagation

- **Data quality:** poor data quality, including noise, incompleteness, or bias, can lead to inaccurate models, as backpropagation learns exactly what it is given.
- **Training duration:** backpropagation often requires extensive training time, which can be impractical when dealing with large networks.
- **Matrix-based complexity:** the matrix operations in backpropagation scale with the network size, which increases the computational demand, and potentially outstripping the available resources.

Loss Function

- Loss functions are one of the most important aspects of neural networks, as they (along with the optimization functions) are directly responsible for fitting the model to the given training data.
- A **loss function** is a function that compares the target and predicted output values; measures how well the neural network models the training data.

Loss Function

- A loss function measures how good a neural network model is in performing a certain task, which in most cases is regression or classification.
- We must minimize the value of the loss function during the backpropagation step in order to make the neural network better.
- The neural network's only objective is to minimize the loss function.

Loss Function: Three Types

- Mean Squared Error Loss Function
- Cross-Entropy Loss Function
- Mean Absolute Percentage Error

Loss Function: MSE

- Mean squared error (MSE) loss function is the sum of squared differences between the entries in the prediction vector and the ground truth vector.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=0}^N (y_i - \hat{y}_i)^2$$

y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\hat{\vec{y}}$

Loss Function: Cross-Entropy

- A loss function that can measure the error between a predicted probability and the label which represents the actual class is called the cross-entropy loss function.

$$\hat{\vec{y}} = \begin{pmatrix} 0 \\ . \\ . \\ . \\ 1 \\ . \\ . \\ . \\ 0 \end{pmatrix} \quad \vec{y} = \begin{pmatrix} 0.52 \\ . \\ . \\ . \\ 0.12 \\ . \\ . \\ . \\ 0.78 \end{pmatrix}$$

Loss Function: Cross-Entropy

$$\mathcal{L}(\theta) = - \sum_{i=0}^N \hat{y}_i \cdot \log(y_i)$$

y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\hat{y}$

Loss Function: MAPE

- We use it to measure the performance of a neural network during demand forecasting tasks.
- Demand forecasting is the area of predictive analytics dedicated to predicting the expected demand for a good or service in the near future.
- Although demand forecasting is also a regression task and the minimization of the MSE loss function is an adequate training goal, this type of loss function to measure the performance of the model during training isn't suitable for demand forecasting.

Loss Function: MAPE

- Let, the MSE loss function gives you a value of 100. Can you tell if this is generally a good result? No, because it depends on the situation. If the prediction of the model is 1000 and the actual ground truth label is 1010, then the MSE loss of 100 would be in fact a very small error and the performance of the model would be quite good.
- However, in the case where the prediction would be 5 and the label is 15, you would have the same loss value of 100 but the relative deviation to the ground-truth value would be much higher than in the previous case.

Loss Function: MAPE

$$MAPE = \frac{100\%}{N} \sum_{i=0}^N \frac{|y_i - \hat{y}_i|}{\hat{y}_i}$$

$$MAPE_1 = 100\% \left| \frac{1000 - 1010}{1000} \right| = 1\%$$

$$MAPE_2 = 100\% \left| \frac{5 - 15}{15} \right| = 66\%$$

Loss Function: Summary

- We only use the cross-entropy loss function in classification tasks when we want the neural network to predict probabilities.
- For regression tasks, when we want the network to predict continuous numbers, we must use the mean squared error loss function.
- We use mean absolute percentage error loss function during demand forecasting to keep an eye on the performance of the network during training time.

Optimization Functions or Optimizers

- The process of minimizing (or maximizing) any mathematical expression is called optimization.
- Optimizers are algorithms or methods used to change the attributes of the neural network such as weights and learning rate to reduce the losses.
- Optimizers are used to solve optimization problems by minimizing the function.

Optimization Functions or Optimizers

- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Mini Batch Stochastic Gradient Descent (MB-SGD)
- SGD with momentum
- Nesterov Accelerated Gradient (NAG)
- Adaptive Gradient (AdaGrad)

- AdaDelta
- RMSprop
- Adam

Optimization Functions or Optimizers

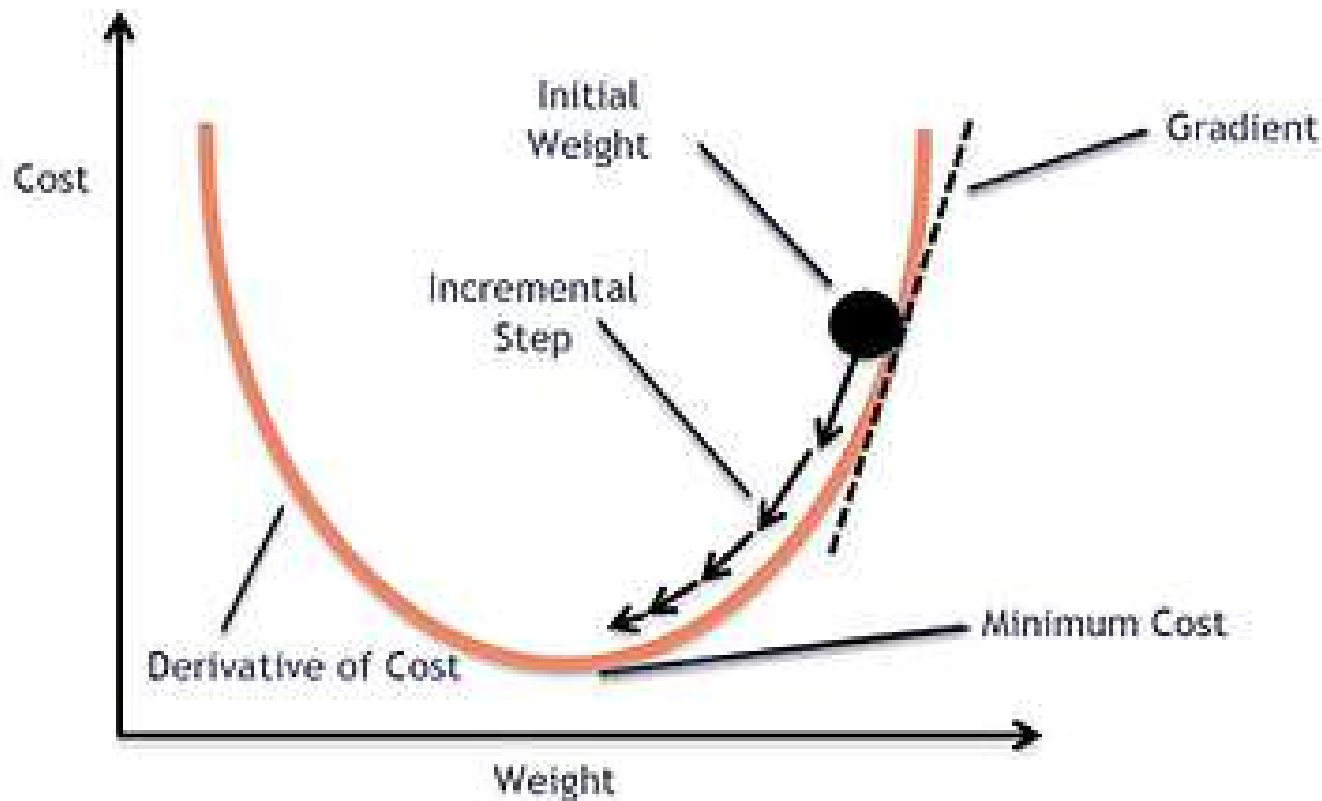
- A gradient measures how much the output of a function changes if you change the inputs a little bit.
- We start by defining the initial parameter's values and from there gradient descent uses calculus to iteratively adjust the values so they minimize the given cost-function.

Optimization Functions or Optimizers

Repeat until convergence {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}



Summary

- Backpropagation